CIVICTRACK

Civic Complaint & Accountability Platform

Project Documentation

01 User Personas

02 System Architecture Diagram

03 User Flow Diagram

04 Component Structure Diagram

05 Threat Model Diagram

06 Threat Model Document

INEZA Sonia · MFURANZIMA Nadette

AUCA Faculty of Information Technology

Web Technology and Internet

SECTION 01

User Personas

Primary User Types & Scenarios

01

USER PERSONAS

CIVICTRACK — Civic Complaint & Accountability Platform

INEZA Sonia & MFURANZIMA Nadette · AUCA Faculty of IT · Web Technology and Internet

These personas represent the two primary user types of CIVICTRACK. Each persona was derived from real civic engagement patterns and informed every design and accessibility decision made during development.

PERSONA 1 — CITIZEN USER

Amahoro Claudine

AC

Primary School Teacher · Citizen User

Age 34 · Kicukiro, Kigali

Tech Comfort: Moderate — uses WhatsApp, mobile banking

"I reported a broken road three times. No one answered. I just want to know someone is listening."

GOALS

- File a complaint about her neighbourhood quickly without bureaucratic friction
- Receive a tracking code so she can prove the complaint was registered
- See status updates without having to call anyone or visit an office
- Stay anonymous when reporting sensitive community disputes

FRUSTRATIONS

- Previous complaint systems required printing forms and visiting offices in person
- No confirmation that her report was received — complaints felt like shouting into a void
- Feared retaliation when reporting against community leaders — anonymity was critical
- Poor mobile experience on previous government platforms

BEHAVIOURS

- Accesses the platform primarily on her Android smartphone during lunch breaks
- Prefers simple, clear language — no technical jargon in the interface
- Will abandon a form if it takes more than 3 minutes to complete
- Shares the platform link with neighbours via WhatsApp groups

SCENARIO

Claudine notices a broken streetlight on her route to school that has caused two accidents. She opens CIVICTRACK on her phone, selects Infrastructure category, sets priority to High, and submits the complaint anonymously. She screenshots the CT-XXXXXXXX tracking code and shares it in her neighbourhood WhatsApp group. A week later she checks the status — it has moved from Pending to Under Review with an authority note. She feels heard for the first time.

PERSONA 2 — ADMINISTRATOR

Nkurunziza Patrick

District Complaints Officer · Administrator

NP

Age 41 · Nyarugenge District Office, Kigali

Tech Comfort: High — uses government systems, Excel, email daily

"I manage 200+ complaints a month. I need to see what is critical, act fast, and have a clear record."

GOALS

- Quickly triage incoming complaints by priority and category across the entire district
- Update complaint statuses and add official authority notes visible to citizens
- Search and filter complaints to identify repeat issues and emerging patterns
- Maintain a verifiable audit trail for every action taken on a complaint

FRUSTRATIONS

- Paper-based logs are lost or misfiled — no searchable history
- Citizens call the office asking for updates he cannot find quickly
- No way to flag critical complaints that need same-day escalation
- Cannot tell which complaints have gone too long without a response

BEHAVIOURS

- Works from a desktop computer at the district office — not primarily mobile
- Processes complaints in batches — morning triage, afternoon updates
- Expects a dashboard that shows him the most important information at a glance
- Needs to export or reference complaint data during monthly reporting meetings

SCENARIO

Patrick starts his Monday morning by opening the Admin Dashboard. The overview shows 12 new Pending complaints since Friday — 2 are marked Critical. He filters by Critical priority, reads the details, adds an authority note to both ("Escalated to infrastructure department — response within 48h"), and changes their status to Under Review. Citizens automatically see the update. He then checks the Users tab to verify a new citizen account that was flagged for suspicious activity.

INEZA Sonia · MFURANZIMA Nadette · AUCA Faculty of Information Technology · WEB TECHNOLOGY AND INTERNET

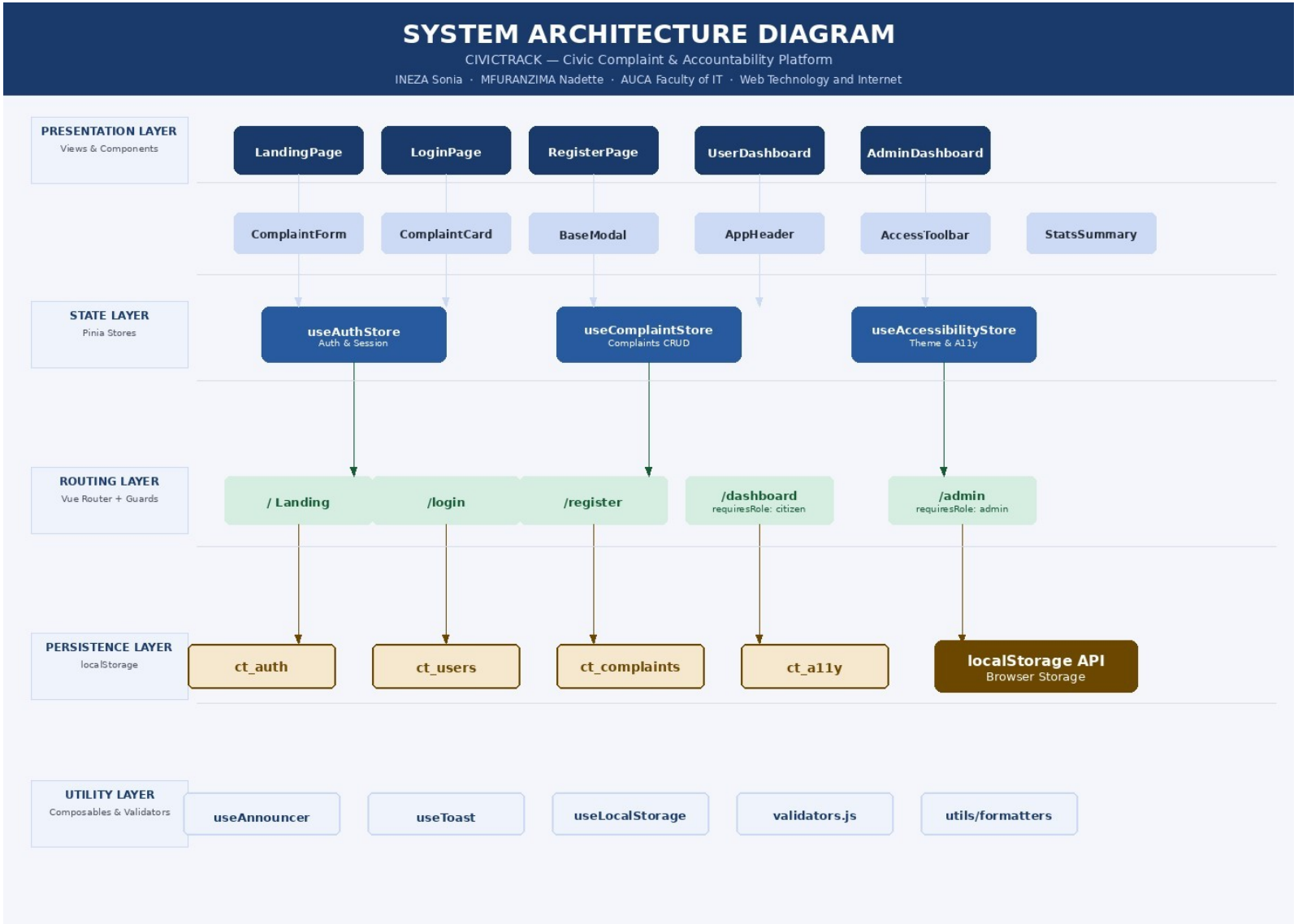
SECTION 02

System Architecture

Layered Architecture Overview

02

SYSTEM ARCHITECTURE



INEZA Sonia · MFURANZIMA Nadette · AUCA Faculty of IT · Web Technology and Internet

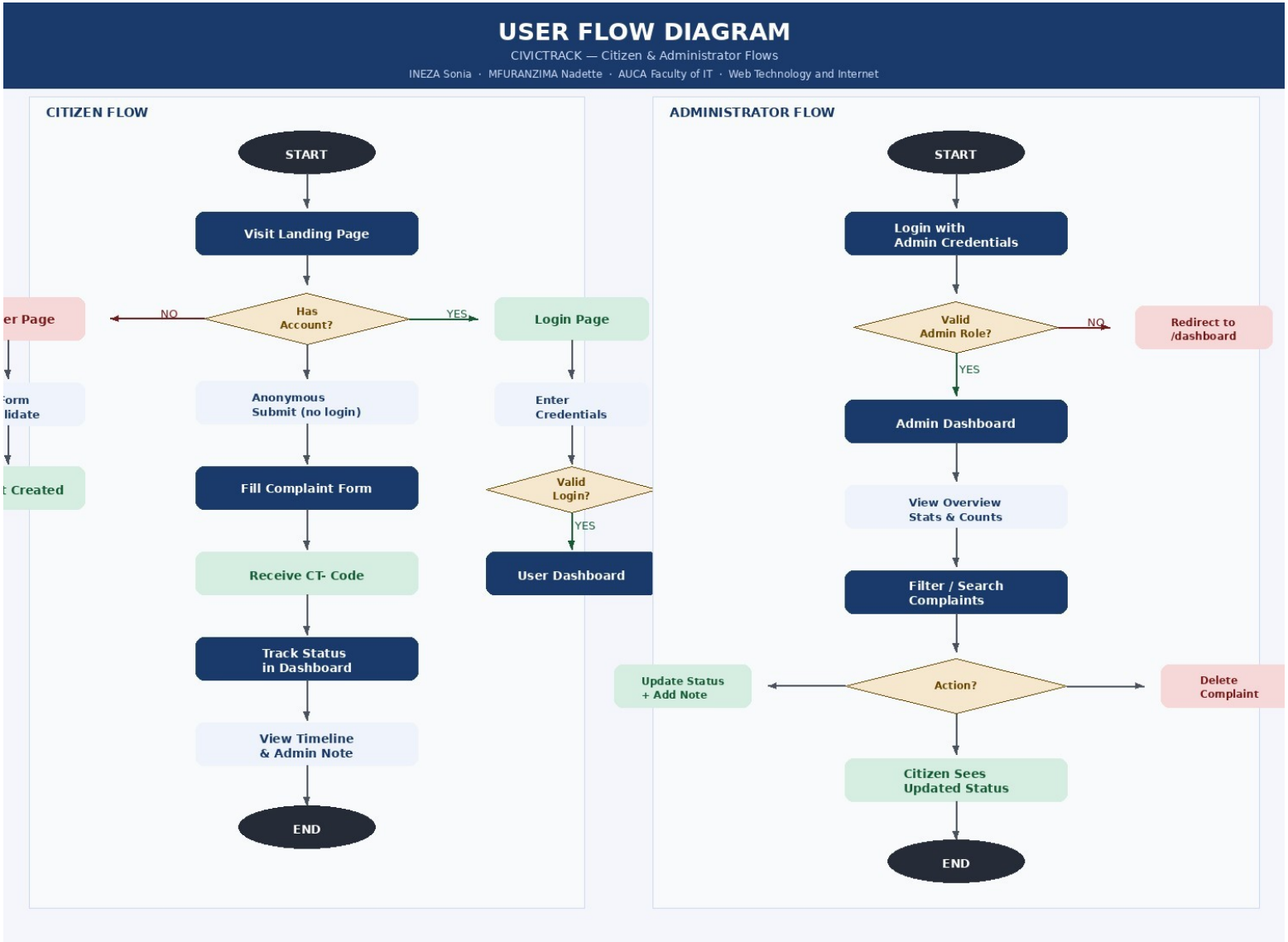
SECTION 03

User Flow Diagram

Citizen & Administrator Flows

03

USER FLOW DIAGRAM



INEZA Sonia · MFURANZIMA Nadette · AUCA Faculty of IT · Web Technology and Internet

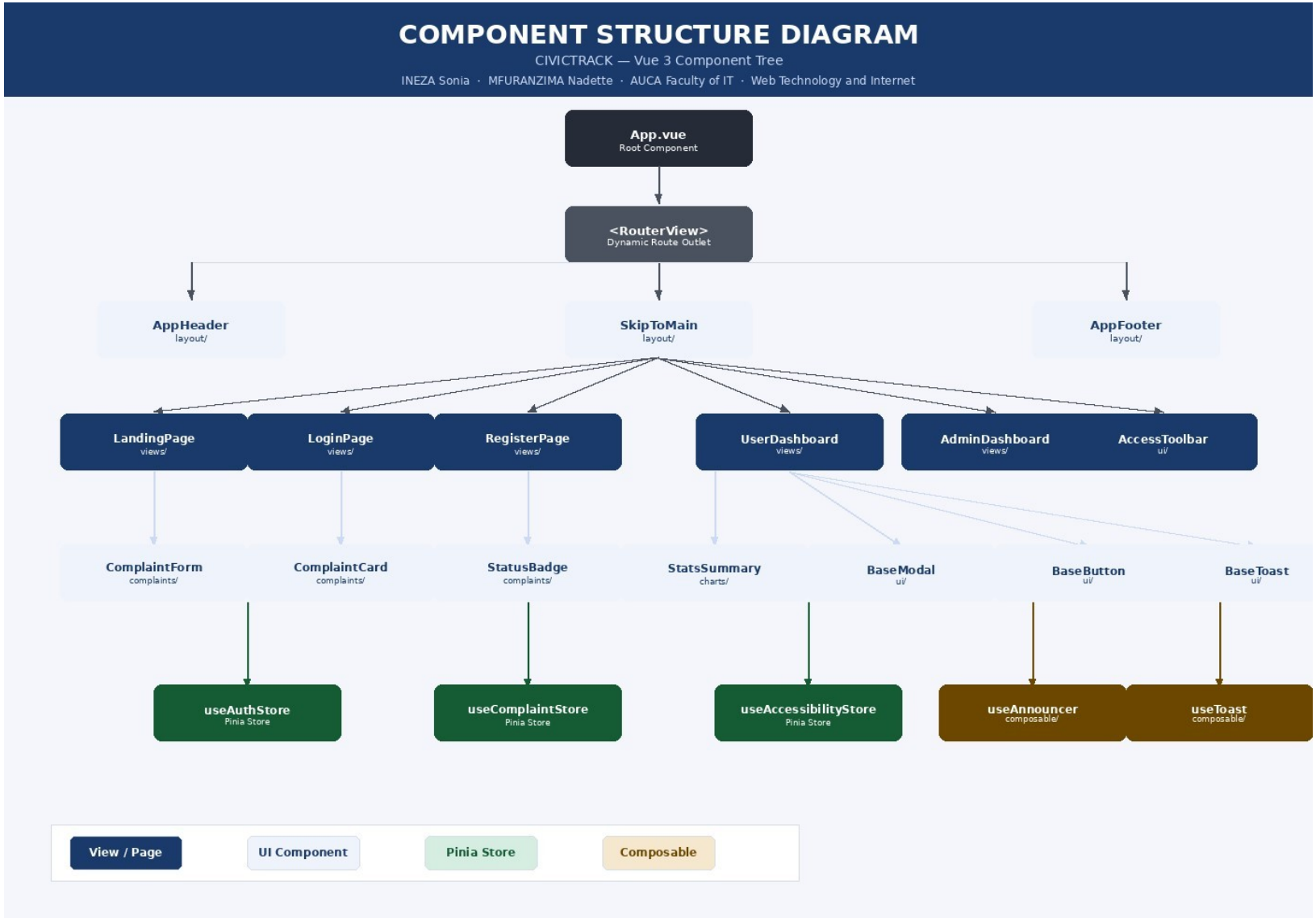
SECTION 04

Component Structure

Vue 3 Component Tree

04

COMPONENT STRUCTURE



INEZA Sonia · MFURANZIMA Nadette · AUCA Faculty of IT · Web Technology and Internet

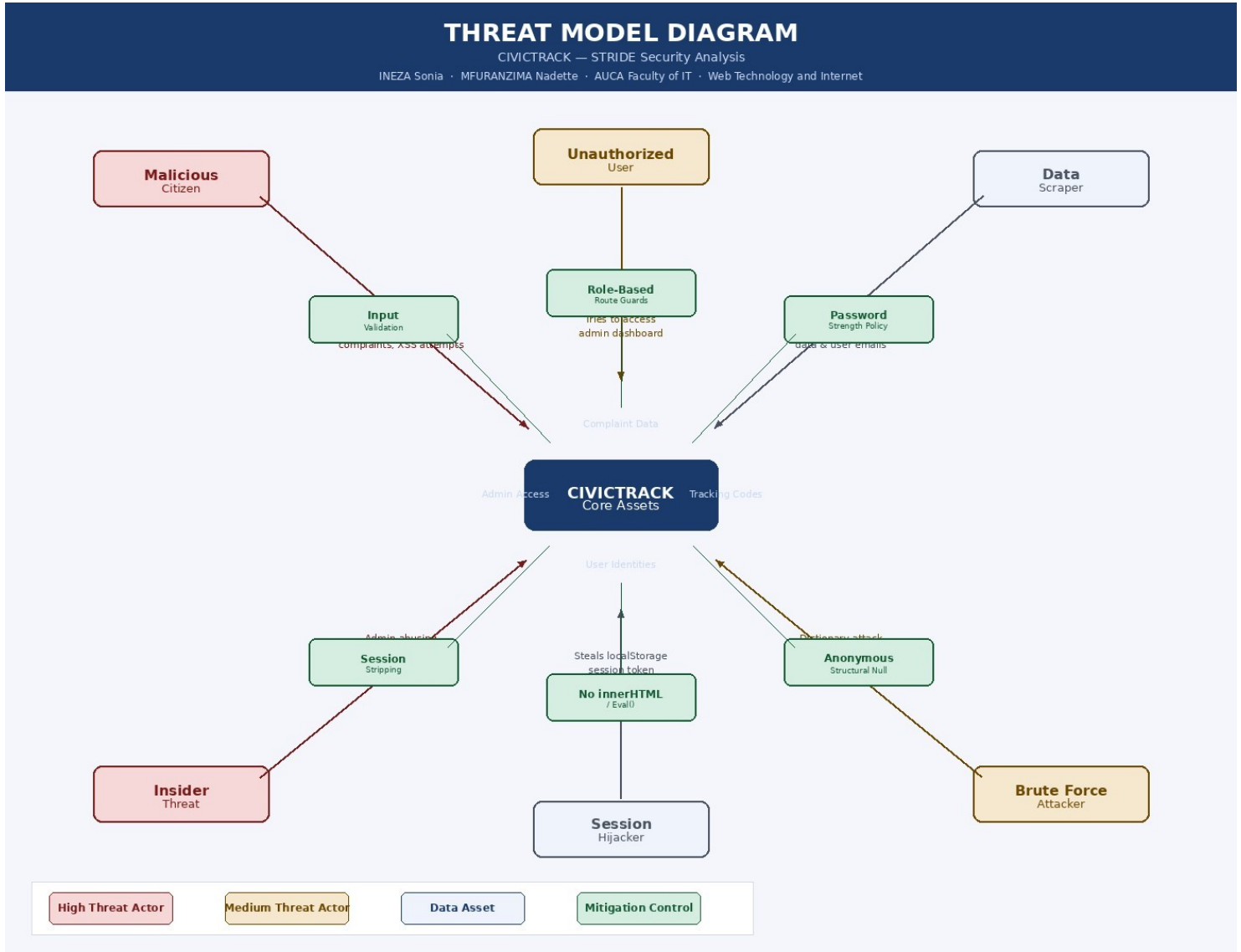
SECTION 05

Threat Model Diagram

STRIDE Security Analysis

05

THREAT MODEL DIAGRAM



INEZA Sonia · MFURANZIMA Nadette · AUCA Faculty of IT · Web Technology and Internet

SECTION 06

Threat Model Document

Secure Design & Threat Analysis

06

THREAT MODEL DOCUMENT

CIVICTRACK — Secure Design & Threat Analysis

INEZA Sonia · MFURANZIMA Nadette · AUCA Faculty of IT · Web Technology and Internet

This document presents the security threat model for CIVICTRACK, a civic complaint platform. It identifies protected assets, potential attackers, threats using the STRIDE framework, mitigation strategies applied, and explains how security was considered at every stage of development.

1. ASSETS TO PROTECT

Asset	Description	Why It Matters
User Identities	Registered citizens: name, email, phone, password	Exposure could enable targeted harassment or identity theft
Complaint Data	All submitted complaints including location, category, description	Sensitive — could reveal personal situations, domestic disputes, or crime reports
Anonymous Submissions	Complaints filed without a user account	If de-anonymised, reporter could face retaliation from authorities or community
Admin Credentials	Administrator login: email and password	Full platform access — compromise means full data exposure and status manipulation
Tracking Codes	CT-XXXXXXXX codes linking complaints to reporters	Could be used to correlate anonymous reporters if intercepted
Session State	localStorage auth token persisting the logged-in user	Theft enables full impersonation without credentials

2. POSSIBLE ATTACKERS

Attacker Profile	Motivation	Technical Capability
Malicious Citizen	Flood system with false complaints to overwhelm admins or discredit platform	Low — uses browser, no coding required
Unauthorized User	Access admin dashboard to alter complaint statuses, delete records	Medium — knows how to manipulate URLs and inspect localStorage
Data Scraper	Harvest citizen contact data for spam, phishing, or surveillance	Medium-High — uses automated browser scripts
Attacker Profile	**Motivation**	**Technical Capability**
Insider Threat (Admin)	Abuse admin privileges to delete legitimate complaints or expose identities	High — already has valid credentials

Session Hijacker	Steal another user's localStorage session to impersonate them	High — requires XSS vulnerability or physical access to device
Brute Force Attacker	Systematically guess admin password to gain full platform control	Low-Medium — uses automated tools

3. THREAT LIST — STRIDE FRAMEWORK

STRIDE Category	Threat	Target Asset	Risk Level
Spoofing	Login with another user's credentials	User identity, session	HIGH
Tampering	Modify localStorage complaint data directly in browser DevTools	Complaint data	MEDIUM
Repudiation	Admin denies updating a complaint status with no audit trail	Timeline history	LOW
Information Disclosure	XSS attack extracts localStorage tokens	Session, user data	HIGH
Information Disclosure	Anonymous reporter re-identified via tracking code	Reporter identity	MEDIUM
Denial of Service	Mass spam submission floods complaint list	Platform usability	MEDIUM
Elevation of Privilege	Citizen navigates directly to /admin URL	Admin dashboard	HIGH
Elevation of Privilege	Malicious script injection via complaint text	All user sessions	HIGH
Brute Force	Repeated login attempts against admin account	Admin credentials	HIGH

4. MITIGATION STRATEGIES

Threat	Mitigation Applied	Location in Code
XSS / Script Injection	Vue 3 template bindings escape all dynamic content by default. No use of v-html, innerHTML, or eval() anywhere in the codebase.	All .vue templates
Elevation of Privilege	Vue Router navigation guards check both authentication AND role before allowing access to /dashboard or /admin. Role mismatch redirects to correct route.	src/router/index.js
Threat	**Mitigation Applied**	**Location in Code**
Spoofing / Session Theft	Password is stripped from session object before writing to localStorage: const { password: _, ...safeUser } = user. Session never contains credentials.	useAuthStore.js — login(), register()
Anonymous Re-identification	When anonymous: true, userId is set to null at the store level — not just hidden in the UI. The data structure itself contains no link to the reporter.	useComplaintStore.js — submit()

Brute Force	Password strength enforced at registration (8+ chars, uppercase, number, special character). Login has simulated delay. (Production: add rate limiting and lockout.)	RegisterPage.vue, validators.js
Input Tampering / Injection	All form fields run through validators.js before any state write. Minimum and maximum lengths enforced. Nothing is submitted raw.	src/utils/validators.js
Repudiation	Every complaint status change appends an immutable entry to the timeline array with timestamp. History is append-only — no overwriting of previous states.	useComplaintStore.js — updateStatus()
Data Exposure	No sensitive data passed in URL parameters. No logging of user data to console. Route guards prevent cross-role data access.	All views and router

5. SECURITY THINKING DURING DEVELOPMENT

We avoided inline HTML injection because Vue's default template syntax ({{ }}) automatically escapes content as text, not HTML. Using v-html would have bypassed this protection, so we deliberately never used it — even for formatted complaint descriptions. This eliminates an entire class of XSS vulnerabilities at the framework level.

We validated user input on every form field because the frontend is the first and only defense in a client-side demo. Each field in ComplaintForm, LoginPage, and RegisterPage runs through named validator functions that return specific error messages. Nothing reaches the Pinia store without passing validation.

We avoided exposing sensitive data by stripping the password field before writing the user object to localStorage. The session only contains what is needed for rendering (name, email, role) — not credentials. This means even if localStorage is inspected in DevTools, no password is visible.

We thought like attackers when designing the anonymous submission feature. A naive implementation might set a "showAuthor: false" flag in the UI. We rejected this because the data still exists. Instead, userId is structurally set to null at the store level — there is no user link to reveal, even if someone queries the raw data.

We acknowledged what we deferred. Passwords are stored plain text in this demo. In production, bcrypt with a cost factor of 12+ is required. There is no server-side rate limiting on login attempts. No Content Security Policy headers are set. These are not oversights — they are documented in the technical documentation with the exact production migration steps.